Monitoring and Missing Telemetry

Day 14 | AWS / AZURE / GCP + OPEN SOURCE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

Monitoring has to detect both observed failure and loss of observation. A failed scrape can produce up equals zero; a missing target can remove the time series instead. Neither proves the user journey is healthy without appropriate independent checks.

The Scenario A dashboard retains old values while a target fails or disappears from discovery.

Two Failures A failed scrape and a missing time series need different checks.

Open-Source Tools Prometheus, Alertmanager, Grafana, node_exporter and blackbox_exporter.

Before you start

A working Prometheus job and rule-loading path; Alertmanager routing; an expected target inventory; an external availability check; notification test recipients.

Diagnostic checks

```
# Inspect Prometheus targets + last scrape.
# Check rule evaluation and Alertmanager.
# Test an external availability probe.
```

Up = 0: Prometheus knows the target, but its scrape fails.

Series Absent: Discovery or labels can remove the target entirely. up == 0 alone may not detect that.

Apply the cause specific fix

Same Check Different Tooling

Aws: CloudWatch metrics and alarms. Configure missing-data behaviour and notification delivery.

Azure: Azure Monitor alerts and Action Groups. Check rule scope and evaluation settings.

Gcp: Cloud Monitoring alerting and uptime checks. Validate notifications and missing-data handling.

Fix The Confirmed Cause

```
groups:
- name: api-availability
  rules:
  - alert: ApiTelemetryUnavailable
    expr: >-
      up{job="api"} == 0
      or absent(up{job="api"})
    for: 2m
    labels:
      severity: critical
```

Prometheus Rule File: Load this file through rule_files and configure Alertmanager routing.

Know The Limit: absent here detects all api series disappearing, not every missing instance in a larger fleet.

Validate the result

```
promtool check rules rules.yaml

# In a lab, fail a scrape then remove all targets.
# Confirm firing AND delivered notification.
```

Test The Whole Path: A firing rule is not proof the on-call notification arrived.

Monitor Monitoring: Use independent checks and watchdogs for the telemetry/alerting system itself.

Practice exercise

Fail a lab target while keeping it discovered, then remove every target for the job. Confirm the rule fires and the test notification is delivered. Test monitoring-system failure separately.

Implementation notes

Plain Prometheus rule file, not a Prometheus Operator PrometheusRule object. Requires a job named api and rule_files registration. Alertmanager receivers/routes are separate. The rule has no annotations but is valid. absent(up{job="api"}) catches total series absence, not an individual disappeared target; per-target missing detection needs an expected inventory/heartbeat design. A scrape failure is not proof the application is down. Add blackbox/user-journey checks and monitor the monitoring infrastructure independently.

Interview question

Why can up equals zero miss a target? If discovery removed the target entirely, there may be no matching series to compare with zero.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1

Official documentation 2